Objetivos da aula:

- Compreender a execução de pipelines de CI no GitHub Actions utilizando runners
- Entender o funcionamento e a estrutura de um arquivo ci.yml
- Diferenciar execução em runner e em container
- Configurar e executar workflows manualmente com workflow_dispatch
- Analisar vantagens do uso de containers para reprodutibilidade e controle de ambiente

Repositório: https://github.com/arleysouza/tpII-2026-1-github-actions

Índice:

i. Introdução ao GitHub Actions e Integração Contínua
ii. Entendendo o arquivo ci.yml com execução em runner
iii. Integração Contínua com Containers

i. Introdução ao GitHub Actions e Integração Contínua

1.1. Motivação: do teste manual à automação

Nas aulas anteriores, foi estabelecido que testes unitários são fundamentais para garantir a qualidade do software. Eles permitem verificar automaticamente se pequenas partes do sistema continuam funcionando corretamente após modificações no código.

Entretanto, surge um problema prático no contexto de desenvolvimento real:

- Quem garante que os testes serão executados sempre?
- O que acontece se um desenvolvedor esquecer de rodá-los?
- Como assegurar que alterações feitas por diferentes pessoas não quebrem o sistema?

Essas questões evidenciam uma limitação importante:

executar testes manualmente não é suficiente em ambientes colaborativos.

Diante disso, surge a necessidade de automatizar esse processo.

1.2. O que é Integração Contínua (CI)

A Integração Contínua (Continuous Integration — CI) é uma prática de engenharia de software que consiste em:

- integrar alterações de código com frequência;
- executar testes automaticamente a cada mudança;
- detectar falhas o mais cedo possível.

Em termos práticos, o fluxo típico é:

1. O desenvolvedor realiza uma alteração no código;
2. Essa alteração é enviada ao repositório;

3. Um sistema automatizado executa:
    - instalação de dependências;
    - build do projeto;
    - execução dos testes;
4. O sistema informa se o código está correto ou se houve falhas.

Esse processo reduz significativamente o risco de introduzir erros no sistema e está diretamente relacionado aos benefícios discutidos anteriormente, como confiabilidade e segurança para refatorações.

1.3. O que é o GitHub Actions

O GitHub Actions é uma ferramenta de automação integrada à plataforma GitHub, utilizada para implementar pipelines de Integração Contínua e Entrega Contínua (CI/CD).

Ele permite definir, por meio de arquivos de configuração, processos automatizados que são executados em resposta a eventos no repositório, como:

- push de código;
- abertura de pull requests;
- criação de releases.

Esses processos automatizados são chamados de **workflows**.

Características principais

O GitHub Actions apresenta algumas características relevantes:

- **Integração nativa com o GitHub**

  Não há necessidade de ferramentas externas para iniciar o uso.
- **Configuração declarativa (YAML)**

  Os fluxos são definidos em arquivos simples dentro do repositório.
- **Execução em ambientes isolados (runners)**

  Cada execução ocorre em uma máquina virtual limpa.
- **Flexibilidade**

  Pode ser utilizado para testes, build, deploy, lint, entre outros.

1.4. Conceitos fundamentais

Para compreender o funcionamento do GitHub Actions, é importante conhecer alguns conceitos básicos:

Workflow

É o processo automatizado como um todo. Exemplo:

- executar testes ao fazer push

Job

É um conjunto de tarefas dentro de um workflow.

- Um workflow pode ter múltiplos jobs
- Jobs podem rodar em paralelo ou sequencialmente

Step

É uma ação individual dentro de um job. Exemplo:

- instalar dependências
- rodar testes

Runner

É a máquina (virtual) onde o workflow é executado.

Exemplo:

- Ubuntu
- Windows
- macOS

1.5. Concorrentes e ferramentas similares

Embora o GitHub Actions seja amplamente utilizado, ele não é a única solução disponível para CI/CD. Existem diversas ferramentas no mercado com propostas semelhantes.

A seguir, destacam-se algumas das principais.

GitLab CI/CD

- Integrado ao GitLab
- Configuração também via YAML
- Muito utilizado em ambientes corporativos

Diferencial: integração completa com todo o ecossistema GitLab.

Jenkins

- Ferramenta open source tradicional
- Extremamente flexível
- Baseada em plugins

Ponto importante:

- Requer configuração e manutenção de servidor

CircleCI

- Plataforma focada em performance
- Muito utilizada em startups

Diferencial:

- Execuções rápidas e paralelização eficiente

Travis CI

- Popular em projetos open source

- Integração simples com GitHub

Azure DevOps

- Solução da Microsoft
- Integração com serviços Azure

1.6. Por que utilizar GitHub Actions neste contexto

Considerando o ambiente acadêmico e o uso de GitHub nos projetos, o GitHub Actions apresenta vantagens claras:

- não exige configuração de infraestrutura;
- possui integração direta com os nossos repositórios no GitHub;
- facilita a visualização de execuções;
- permite rápida adoção com baixo custo cognitivo.

Além disso, ele se encaixa perfeitamente no fluxo já estudado:

- testes unitários com Jest;
- aplicações em NestJS com dependências isoladas;
- agora, execução automática desses testes.

ii. Entendendo o arquivo ci.yml com execução em runner

2.1. Introdução

No capítulo anterior, foi apresentado o conceito de Integração Contínua e o papel do GitHub Actions como ferramenta de automação dentro do repositório. A partir deste ponto, o objetivo passa a ser compreender, de forma concreta, como essa automação é descrita em um arquivo de workflow.

No GitHub Actions, os fluxos automatizados são definidos por arquivos escritos em YAML, normalmente armazenados no diretório:

```
.github/workflows/
```

Utilizaremos como exemplo o arquivo `ci.yml`, responsável por executar verificações automáticas no projeto sempre que ocorrerem alterações relevantes no repositório.

O primeiro cenário estudado será a execução usando um **runner**, isto é, uma máquina virtual disponibilizada pelo próprio GitHub para rodar o workflow. Nesse modelo, o job é executado diretamente em um ambiente hospedado, sem a necessidade de definir um contêiner manualmente.

2.2. Arquivo base utilizado

O que está sendo testado

O workflow analisado será o seguinte (arquivo `.github/workflows/ci-runner.yml`):

```yaml
name: CI
```

```yaml
on:
  push:
    branches: ["main"]
  pull_request:
    branches: ["main"]

concurrency:
  # Garante apenas 1 execução por branch/workflow
  group: ${{ github.workflow }}-${{ github.ref }}
# Cancela runs antigos se um novo push for feito
  cancel-in-progress: true

jobs:
  test:
    runs-on: ubuntu-latest
    permissions:
      contents: read

    steps:
      - name: Checkout do codigo
        uses: actions/checkout@v6

      - name: Configura Node.js
        uses: actions/setup-node@v6
        with:
          node-version: 22.x
          cache: npm

      - name: Instala dependencias
        run: npm ci

      - name: Executa lint
        run: npm run lint

      - name: Gera build
        run: npm run build

      - name: Executa testes com cobertura
        run: npm run test:cov -- --runInBand

      - name: Upload coverage report
        if: success() || failure()
        uses: actions/upload-artifact@v6
        with:
          name: coverage-report
          path: coverage
```

Esse arquivo descreve um fluxo de Integração Contínua relativamente completo, pois não executa apenas testes. Ele também realiza lint, build e upload do relatório de cobertura.

2.3. Estrutura geral do workflow

Antes de analisar cada linha, convém perceber que o arquivo está organizado em grandes blocos:

1. identificação do workflow;
2. eventos que disparam a execução;
3. política de concorrência;
4. definição dos jobs;
5. passos executados dentro de cada job.

Essa organização é importante porque o GitHub Actions interpreta o arquivo exatamente nessa lógica: primeiro ele identifica quando deve rodar, depois decide como organizar a execução, e por fim executa os comandos especificados.

2.4. Nome do workflow

O arquivo começa com:

```
name: CI
```

Essa linha define o nome do workflow que será exibido na aba **Actions** do GitHub.

O nome não altera o comportamento técnico da execução, mas tem função organizacional. Ele permite identificar com clareza qual pipeline foi executado. Em projetos maiores, pode haver mais de um workflow, como:

- CI
- Deploy
- Release
- Lint
- Security Scan

Neste caso, o nome `CI` indica que se trata do pipeline principal de integração contínua.

2.5. Evento de disparo: on

Em seguida, aparece o bloco:

```
on:
  push:
    branches: ["main"]
  pull_request:
    branches: ["main"]
```

Esse trecho define **quando** o workflow deve ser executado.

Execução em push

```
  push:
    branches: ["main"]
```

Aqui o workflow será disparado quando houver envio de código para a branch `main`.

Isso significa que, sempre que alguém fizer um push para a branch principal, o GitHub Actions iniciará automaticamente o processo de CI.

Execução em `pull_request`

```
pull_request:
  branches: ["main"]
```

Neste caso, o workflow também será executado quando um pull request tiver como destino a branch main.

Esse comportamento é muito importante porque permite validar o código **antes da fusão**. Em outras palavras, o projeto pode verificar automaticamente se o código submetido em um pull request:

- compila corretamente;
- respeita as regras de lint;
- passa nos testes.

Importante, o GitHub Actions não serve apenas para "testar depois", mas também para **avaliar mudanças antes da integração ao ramo principal**.

2.6. Controle de concorrência

O próximo bloco é:

```
concurrency:
  group: ${{ github.workflow }}-${{ github.ref }}
  cancel-in-progress: true
```

Esse trecho define uma política de concorrência para evitar execuções redundantes.

O campo `group`

```
group: ${{ github.workflow }}-${{ github.ref }}
```

Esse grupo é formado pela combinação de:

- nome do workflow;
- referência do Git atual, isto é, a branch ou referência associada à execução.

Na prática, isso significa que execuções do mesmo workflow na mesma branch pertencem ao mesmo grupo.

O campo `cancel-in-progress`

```
cancel-in-progress: true
```

Aqui o GitHub Actions recebe a instrução de cancelar execuções anteriores ainda em andamento quando uma nova execução for iniciada no mesmo grupo.

Por que isso é útil

Imagine a situação em que um programador faz vários pushes consecutivos para a branch main ou para uma branch de trabalho vinculada a um pull request. Sem controle de concorrência, o GitHub poderia executar vários pipelines obsoletos, consumindo tempo e recursos.

Com essa configuração:

- a execução antiga é cancelada;
- apenas a execução mais recente continua.

Isso melhora a eficiência e evita que a equipe perca tempo analisando resultados já ultrapassados.

Atenção: Considere que Ana e Carlos tenham acesso ao repositório

Se Ana faz push em main e, antes de terminar, Carlos faz outro push em main, a execução da Ana será cancelada porque ambas caem no mesmo grupo.

Para isolarmos por autor, teríamos que incluir algo como `${{ github.actor }}` no group.

```
concurrency:
  # Isola execucoes por workflow, branch e autor
  group: ${{ github.workflow }}-${{ github.ref }}-${{ github.actor }}
  # Cancela runs antigos apenas do mesmo autor no mesmo grupo
  cancel-in-progress: true
```

2.7. Definição do job

O bloco seguinte é:

```
jobs:
  test:
```

Aqui começa a definição dos jobs do workflow.

O nome `test` é o identificador interno do job. Ele representa o processo que será executado. Embora o nome seja `test`, o job faz mais do que testar: ele também faz checkout, configura Node.js, instala dependências, roda lint, build e testes com cobertura.

Em YAML, o identificador do job não precisa ser igual ao nome que será mostrado ao usuário. Ele serve como chave estrutural dentro do arquivo.

```
jobs:
  test:
    name: Lint, Build e Testes
    runs-on: ubuntu-latest
    permissions:
      contents: read
```

Aqui o identificador continua sendo `test`, mas o nome visível no Actions será `Lint, Build e Testes`.

2.8. Execução em runner com runs-on

No Capítulo 2, este é o ponto mais importante:

```
runs-on: ubuntu-latest
```

Esse comando informa que o job será executado em um **runner hospedado pelo GitHub**, usando a imagem mais recente do Ubuntu.

O que é um runner

Um runner é a máquina que executa o workflow.

No modelo mais simples, o GitHub disponibiliza ambientes prontos, como:

- Ubuntu
- Windows
- macOS

Neste exemplo, foi escolhido:

```
ubuntu-latest
```

Ou seja, quando o workflow iniciar, o GitHub criará uma máquina virtual com Ubuntu, executará os passos definidos e depois descartará esse ambiente ao final da execução.

2.9. Permissões do job

Na sequência, aparece:

```
permissions:
  contents: read
```

Esse trecho define as permissões concedidas ao job em relação ao repositório.

Neste caso, a permissão é apenas de leitura do conteúdo do repositório.

Interpretação

O pipeline precisa ler os arquivos do projeto para:

- fazer checkout;
- instalar dependências;
- compilar;
- testar.

Como não há necessidade de alterar o repositório, publicar release ou fazer escrita em conteúdo, a permissão de leitura é suficiente.

Esse é um bom exemplo de princípio de menor privilégio: conceder apenas o necessário para a execução.

2.10. Os passos do job: `steps`

A partir daqui, o workflow descreve uma sequência de ações:

```
steps:
```

Cada item da lista representa uma etapa executada em ordem. Se uma etapa falhar, o comportamento padrão é interromper o job, exceto quando houver condições específicas.

2.11. Checkout do código

Primeiro passo:

```
- name: Checkout do codigo
  uses: actions/checkout@v6
```

Esse passo baixa o conteúdo do repositório para o runner.

Por que ele é necessário

Quando o runner é iniciado, ele não possui automaticamente o código do projeto. É preciso clonar o repositório para que os comandos seguintes consigam acessar arquivos como:

- package.json
- src/
- test/
- arquivos de configuração

A action `actions/checkout` realiza exatamente essa tarefa.

Observação didática importante

Sem esse passo, os comandos `npm ci`, `npm run build` e `npm run test:cov` não teriam sobre o que atuar, porque o código do projeto não estaria presente no ambiente.

2.12. Configuração do Node.js

O segundo passo é:

```
- name: Configura Node.js
  uses: actions/setup-node@v6
  with:
    node-version: 22.x
    cache: npm
```

Esse trecho instala e configura o Node.js no runner.

node-version

```
node-version: 22.x
```

Define a versão do Node.js que será utilizada na execução.

Isso é importante porque projetos JavaScript e TypeScript podem ter dependências ou comportamentos sensíveis à versão do ambiente. Ao fixar a versão, o pipeline se torna mais previsível.

cache: npm

```
cache: npm
```

Habilita cache para dependências do npm, com o objetivo de acelerar execuções futuras.

Na prática, o GitHub Actions reaproveita partes já instaladas anteriormente quando possível, reduzindo o tempo do workflow.

Importante

Esse passo reforça uma ideia importante: o pipeline não deve depender do computador do desenvolvedor. Ele precisa montar seu próprio ambiente de execução, de forma controlada e reprodutível.

2.13. Instalação de dependências com npm ci

O próximo passo é:

```
- name: Instala dependencias
  run: npm ci
```

Aqui o workflow executa um comando de terminal diretamente no runner.

Diferença entre `run` e `uses`

- `uses` emprega uma action pronta;
- `run` executa um comando shell.

Por que `npm ci` e não `npm install`

O comando `npm ci` é mais adequado para pipelines de CI porque:

- usa o `package-lock.json` de forma estrita;
- instala dependências de forma reprodutível;
- é mais apropriado para ambientes automatizados.

Do ponto de vista didático, este é um excelente momento para reforçar que CI deve privilegiar previsibilidade. O objetivo não é "instalar qualquer versão compatível", mas sim reproduzir com fidelidade o ambiente esperado pelo projeto.

2.14. Execução do lint

Em seguida:

```
- name: Executa lint
  run: npm run lint
```

Esse passo executa o lint do projeto.

O que o lint verifica

O lint analisa o código em busca de:

- problemas de estilo;
- más práticas;
- padrões inconsistentes;
- possíveis erros simples.

Dependendo da configuração do projeto, falhas de lint podem interromper o pipeline mesmo antes do build ou dos testes.

Importância no CI

Colocar lint no pipeline significa automatizar a verificação de padronização do código. Isso é especialmente útil em trabalhos em grupo, pois garante que o repositório mantenha consistência mínima entre contribuições diferentes.

2.15. Geração do build

Depois temos:

```
- name: Gera build
  run: npm run build
```

Esse passo verifica se o projeto consegue ser compilado ou empacotado corretamente.

No contexto de projetos TypeScript ou NestJS, isso é muito importante, porque um código pode até passar em certas verificações locais, mas ainda falhar na transpilação ou na geração do build.

O que esse passo garante

Ele ajuda a responder a seguinte pergunta:

> O projeto está em um estado que permite ser construído corretamente?

Portanto, o pipeline não está apenas testando comportamento, mas também verificando integridade estrutural do sistema.

2.16. Execução de testes com cobertura

O próximo passo é:

```
- name: Executa testes com cobertura
  run: npm run test:cov -- --runInBand
```

Aqui ocorre a execução dos testes automatizados com geração de cobertura.

`npm run test:cov`

Esse comando normalmente executa os testes e produz um relatório com informações sobre cobertura de código, isto é, quais partes do sistema foram exercitadas pelos testes.

`--runInBand`

Esse parâmetro instrui o Jest a rodar os testes em sequência, em vez de paralelizar.

Por que isso pode ser útil

Em alguns projetos, essa opção ajuda a evitar problemas de concorrência, consumo excessivo de memória ou comportamentos imprevisíveis no ambiente de CI.

Do ponto de vista didático, vale destacar que:

- os testes continuam sendo os mesmos da Aula 8 e da Aula 9;
- o que muda agora é que eles são executados automaticamente pelo pipeline.

Assim, a disciplina avança da escrita de testes para a **automação sistemática da validação do software**.

2.17. Upload do relatório de cobertura

O último passo é:

```
- name: Upload coverage report
  if: success() || failure()
```

```
        uses: actions/upload-artifact@v6
        with:
          name: coverage-report
          path: coverage
```

Esse trecho envia a pasta `coverage` como artefato da execução.

O que é um artefato

Um artefato é um arquivo ou conjunto de arquivos gerado pelo workflow e armazenado pelo GitHub para consulta posterior.

Neste caso, o artefato contém o relatório de cobertura dos testes.

Condição `if: success() || failure()`

```
        if: success() || failure()
```

Essa condição faz com que o upload ocorra tanto em caso de sucesso quanto em caso de falha do job.

Isso é útil porque, mesmo quando algo falha, ainda pode ser interessante analisar os arquivos gerados até aquele ponto.

`path: coverage`

```
          path: coverage
```

Indica que a pasta `coverage` será enviada.

Valor didático

Esse passo mostra que um workflow não serve apenas para "dar certo ou errado". Ele também pode produzir evidências e relatórios úteis para análise do projeto.

2.18. Fluxo completo de execução

Considerando o arquivo inteiro, o fluxo do job é o seguinte:

1. o GitHub detecta um push ou pull_request para main;
2. cria um runner com Ubuntu;
3. baixa o código do repositório;
4. instala e configura Node.js;
5. instala dependências;
6. executa lint;
7. gera build;
8. executa testes com cobertura;
9. envia o relatório de cobertura como artefato.

Essa sequência representa um pipeline de CI bastante representativo de ambientes profissionais.

iii. Integração Contínua com Containers

3.1. Introdução

No capítulo anterior, foi apresentado o uso do GitHub Actions com execução baseada em runner, no qual o ambiente é configurado dinamicamente durante o workflow. Esse modelo é suficiente para muitos cenários e tem como principal vantagem a simplicidade.

Entretanto, em ambientes profissionais, é comum a necessidade de maior controle sobre o ambiente de execução. Para isso, utiliza-se a execução de pipelines dentro de **containers**.

Neste capítulo, será explorado esse segundo modelo de execução, no qual o ambiente já vem previamente definido por uma imagem, tornando o processo mais previsível e reprodutível.

3.2. Organização dos workflows no projeto

Para fins didáticos, o projeto da disciplina utiliza dois arquivos distintos de CI:

ci-runner.yml

- Executado automaticamente
- Disparado por eventos de push e pull request:

```
on:
  push:
    branches: ["main"]
  pull_request:
    branches: ["main"]
```

Esse arquivo representa o fluxo principal de integração contínua, apresentado no capítulo anterior.

ci-container.yml

- Executado manualmente
- Focado em demonstrar execução com containers

```
on:
  workflow_dispatch:
```

Essa separação é intencional e possui valor pedagógico:

- evita complexidade excessiva no fluxo principal;
- permite experimentação controlada;
- facilita a comparação entre abordagens.

3.3. Execução manual com `workflow_dispatch`

O evento:

```
on:
  workflow_dispatch:
```

indica que o workflow será executado manualmente.

Na interface do GitHub, isso aparece como um botão **“Run workflow”**, permitindo ao usuário iniciar a execução sob demanda.

Importância didática

Esse tipo de execução é útil para:

- testar configurações sem impactar o pipeline principal;
- validar mudanças no ambiente;
- experimentar diferentes configurações de CI.

3.4. Arquivo base: `ci-container.yml`

O workflow utilizado neste capítulo é o seguinte:

```yaml
name: CI Container

on:
  workflow_dispatch:

concurrency:
  group: ${{ github.workflow }}-${{ github.ref }}
  cancel-in-progress: true

jobs:
  test:
    runs-on: ubuntu-latest
    container:
      image: node:22-bookworm
    permissions:
      contents: read

    steps:
      - name: Checkout do codigo
        uses: actions/checkout@v6

      - name: Exibe versoes do ambiente
        run: |
          node --version
          npm --version

      - name: Instala dependencias
        run: npm ci

      - name: Executa lint
        run: npm run lint

      - name: Gera build
        run: npm run build

      - name: Executa testes com cobertura
        run: npm run test:cov -- --runInBand
```

```yaml
    - name: Upload coverage report
      if: success() || failure()
      uses: actions/upload-artifact@v6
      with:
        name: coverage-report-container
        path: coverage
```

3.5. Conceito de execução com container

O ponto central do workflow está na diretiva:

```yaml
container:
  image: node:22-bookworm
```

Essa linha define que o job será executado dentro de um container baseado na imagem `node:22-bookworm`.

Interpretação

- o runner (`ubuntu-latest`) ainda é utilizado;
- porém, os comandos não são executados diretamente nele;
- em vez disso, são executados dentro do container.

3.6. Papel do runner e do container

É fundamental distinguir os dois elementos:

Runner

- máquina virtual fornecida pelo GitHub
- responsável por iniciar o job

Container

- ambiente isolado dentro do runner
- onde o código realmente é executado

Síntese

O runner hospeda o container, e o container executa o workflow.

3.7. A imagem `node:22-bookworm`

A imagem utilizada:

```yaml
image: node:22-bookworm
```

é uma imagem oficial do Node.js baseada em Debian (Bookworm).

Ela já inclui:

- Node.js versão 22;
- npm configurado;
- sistema operacional base pronto para uso.

Consequência prática

Não é necessário utilizar:

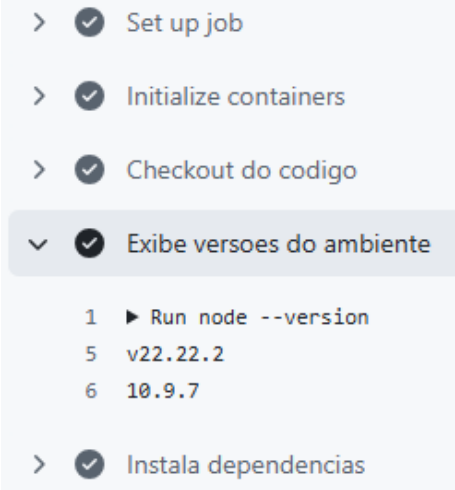
```
actions/setup-node@v6
```

pois o ambiente já está preparado.

3.8. Verificação do ambiente

O workflow inclui o passo:

```
- name: Exibe versoes do ambiente
  run: |
    node --version
    npm --version
```



Objetivo

Esse passo permite verificar explicitamente:

- versão do Node.js;
- versão do npm.

Valor didático

Esse tipo de verificação é importante para:

- validar que o container está sendo utilizado corretamente;
- comparar com o ambiente do runner;
- reforçar a ideia de controle do ambiente.

3.9. Execução dos passos do pipeline

Após o setup inicial, o pipeline segue a mesma lógica do capítulo anterior:

1. Checkout do código

```
uses: actions/setup-node@v6
```

Baixa o repositório para o container.

2. Instalação de dependências

```
run: npm ci
```

Instala dependências de forma reprodutível.

3. Lint

```
run: npm run lint
```

Verifica qualidade e padronização do código.

4. Build

```
run: npm run build
```

Garante que o projeto pode ser compilado corretamente.

5. Testes com cobertura

```
run: npm run test:cov -- --runInBand
```

Executa testes e gera relatório de cobertura.

6. Upload de artefato

```
uses: actions/upload-artifact@v6
```

Armazena o relatório gerado para consulta posterior.

3.10. Comparação com o modelo baseado em runner

A principal diferença entre os dois modelos está no provisionamento do ambiente:

Aspecto	Runner	Container
Ambiente	Configurado durante execução	Pré-configurado
Node.js	Instalado via action	Já presente
Reprodutibilidade	Média	Alta
Complexidade	Baixa	Média

3.11. Vantagens do uso de containers

Reprodutibilidade

Ambiente idêntico em todas as execuções.

Isolamento

Execução independente de configurações externas.

Portabilidade

Mesmo ambiente pode ser usado localmente e em produção.

Previsibilidade

Redução de erros relacionados a ambiente.

3.12. Quando utilizar containers

O uso de containers é recomendado quando:

- há dependências específicas de ambiente;
- é necessário garantir consistência entre ambientes;
- o projeto possui maior complexidade;
- deseja-se aproximar CI de produção.